

PROJECTGENAI

Node Name	Icon/Type	What It Does (Short & Simple)
When chat message received	Trigger	The Entrance: It opens a user-friendly chat window. It catches the project idea typed by the user (e.g., <i>"Food truck tracking app"</i>) and kicks off the background workflow.
Google Gemini Chat Model	AI Model	The Brain: Connected directly to the free-tier Gemini API. It acts as the core intelligence engine that powers the chat trigger and all 4 agents.
AI Agent (Product Manager)	Agent 1	The Strategist: Takes the user's initial idea and breaks it down into core features, defining the overall scope, goals, and problem statement.
AI Agent1 (Market Researcher)	Agent 2	The Analyst: Reviews the product scope to identify the target audience, analyze competitors, and build data-driven market insights.
AI Agent2 (System Architect)	Agent 3	The Engineer: Takes all the features and insights to design the technical foundation selecting the exact database, tech stack, and modular system layout.

PROJECTGENAI

Node Name	Icon/Type	What It Does (Short & Simple)
AI Agent3 (Technical Writer)	Agent 4	The Editor: Gathers the collective output of the first 3 agents, filters out conversational text, and compiles it into a beautiful, formal Markdown document.
Convert to File	Utility	The Packager: Instantly converts the raw text from the Technical Writer into a downloadable, workspace-ready file named Project_Proposal.md.

The screenshot displays the ProjectGenAI workflow interface. At the top, a navigation bar includes 'Personal', 'My workflow 2', and 'Add tag'. Below this, a sidebar on the left contains 'Overview' and 'Chat'. The main workspace shows a workflow diagram with nodes: 'When chat message received', 'AI Agent', 'AI Agent1', 'AI Agent2', 'AI Agent3', and 'Convert to File'. The 'Convert to File' node is highlighted, showing its output: 'Project_Proposal.md' (5.92 KB). The bottom section features a 'Chat' window with a message about a mobile app for tracking street food trucks, a 'Log' window showing the execution of the agents, and an 'OUTPUT' window displaying the generated file details.

Workflow Diagram:

```
graph LR; Start[When chat message received] --> AI_Agent[AI Agent]; AI_Agent --> AI_Agent1[AI Agent1]; AI_Agent1 --> AI_Agent2[AI Agent2]; AI_Agent2 --> AI_Agent3[AI Agent3]; AI_Agent3 --> Convert_to_File[Convert to File];
```

Chat Window:

A mobile app for tracking local street food trucks in real-time

{ "action": "sendMessage", "sessionId": "ea4468badc3a4523a1817f9d88b583b7", "chatInput": "A mobile app for tracking local street food trucks in real-time" }

A mobile app for tracking local street food trucks in real-time

<Empty response>

Type message, or press 'up' for previous one

Log Window:

Success in 52.4% 7688 Tokens

- AI Agent
- Google Gemini Chat Model
- AI Agent1
- Google Gemini Chat Model
- AI Agent2
- Google Gemini Chat Model
- AI Agent3
- Google Gemini Chat Model
- Convert to File

OUTPUT Window:

data

File Name: Project_Proposal.md

File Extension: md

Mime Type: text/plain

File Size: 5.92 KB

View Download

PROJECTGENAI

OUTPUT:

Project_Proposal.md

Formal Project Proposal: Intelligent Inventory & Culinary Orchestration System

****Date:**** May 22, 2024

****Subject:**** Technical Blueprint for an AI-Driven Vision & Recipe Management Platform

****Prepared by:**** Senior Software Architecture Team

1. Executive Summary & Problem Statement

****Executive Summary****

The proposed system is a next-generation "Human-in-the-Loop" inventory management platform that leverages Computer Vision (CV) and Large Language Models (LLMs) to automate household resource tracking and meal planning. Unlike traditional inventory apps that require manual entry, this architecture utilizes an asynchronous, reactive pipeline to transform photos of refrigerators and pantries into actionable data, providing real-time recipe recommendations and waste analytics.

****Problem Statement****

PROJECTGENAI

The primary barrier to digital inventory management is **input friction**. Manual logging is unsustainable for users. Conversely, fully automated Computer Vision systems often suffer from "black box" errors—misidentifying items without a mechanism for user correction. Furthermore, high-performance AI inference presents significant challenges regarding latency, API costs, and data privacy.

2. Recommended Technology Stack & Infrastructure

The architecture follows a **Reactive Micro-services** pattern, designed for high availability and modular scalability.

Component	Technology	Role
	---	---
Cloud Provider	AWS (Amazon Web Services)	Primary infrastructure host.
Orchestration	AWS Step Functions	Manages the complex state machine for image processing.
Compute	Amazon EKS (Kubernetes) & Lambda	EKS for heavy inference; Lambda for API glue logic.
Vision Model	Amazon Rekognition / Custom YOLO	Object detection and bounding box generation.
Reasoning Model	Claude 3.5 Sonnet / GPT-4o	Recipe tailoring and complex semantic mapping.

PROJECTGENAI

| ****Communication**** | Firebase (FCM) | Real-time push notifications for processing updates. |

| ****API Gateway**** | Kong or AWS API Gateway | Rate limiting, authentication, and request routing. |

3. Core System Architecture & API Layout

****The Architectural Pattern: CQRS****

To ensure the heavy lifting of image processing doesn't degrade the user experience, we utilize ****Command Query Responsibility Segregation (CQRS)****.

- * ****Command Side:**** Handles asynchronous image uploads and CV/LLM processing.

- * ****Query Side:**** Optimized for sub-millisecond retrieval of inventory and recipes via Redis.

****Key API Endpoints****

****A. Inventory Management API****

- * ``POST /v1/scans``: Returns a pre-signed S3 URL and a ``job_id``. Initiates the asynchronous pipeline.

- * ``GET /v1/scans/{job_id}``: Polls for the status of the vision pipeline.

PROJECTGENAI

* `PUT /v1/inventory/verify`: ****Critical Path.**** Allows the user to confirm or edit AI-detected items before they are committed to the permanent record.

****B. Intelligence & Recipes API****

* `GET /v1/recipes/recommend`: Utilizes semantic search to match current inventory against 50k+ recipes.

* `POST /v1/recipes/{id}/consume`: An atomic transaction that decrements inventory, updates caloric tracking, and logs activity history.

****C. Analytics API****

* `GET /v1/analytics/waste-report`: Analyzes the delta between `detected\_at` and `expires\_at` to provide insights on purchasing habits.

4. Database Schema Strategy

We employ a ****Polyglot Persistence**** strategy, ensuring the right tool is used for the specific data requirement.

****A. PostgreSQL (The Relational Source of Truth)****

Focuses on data integrity and relational mapping.

PROJECTGENAI

- * ``inventory_items``: Tracks ``status`` (detected | verified | consumed) and ``image_reference_url``.
- * ``ingredients_master``: A global taxonomy of items containing nutritional data and average shelf life.

B. Pinecone / pgvector (The Semantic Layer)

- * `**`recipe_embeddings`**`: Stores vector representations of recipes. Instead of asking an LLM to "invent" a recipe (which is hallucination-prone), we perform a semantic search to find existing recipes that match the user's current inventory vectors.

C. Redis (The Acceleration Layer)

- * `**`session_cache`**`: Stores the immediate "Fridge State" for fast UI rendering.
- * `**`idempotency_keys`**`: Ensures that a double-tap on "Consume Item" does not result in duplicate deductions.

5. Risk Mitigation & Implementation Phases

Risk Mitigation Strategies

- * `**Cost Control (Tiered Inference)**` We implement a two-tier AI strategy. Low-level categorization is handled by cheaper models (Llama 3); premium models (GPT-4o) are reserved only for final recipe personalization.

PROJECTGENAI

* **Latency (Shadow Processing):** The system begins "pre-warming" recipe suggestions the moment the first three ingredients are identified, significantly reducing perceived wait time.

* **Data Privacy:** S3 buckets utilize a 24-hour lifecycle policy to automatically purge raw images. A pre-processing Lambda strips PII (faces/identifiable background info) using blur filters before cloud processing.

* **Accuracy (Confidence Thresholds):**

* *>90% Confidence:* Automatic inventory addition.

* *50-89% Confidence:* Flagged for "User Review."

* *<50% Confidence:* Trigger OCR for specific label scanning or discard.

Implementation Phases

1. **Phase I (Foundation):** Setup S3/Step Function pipeline and basic PostgreSQL schema. Implement YOLO-based object detection.

2. **Phase II (Intelligence):** Integrate Pinecone for recipe vector search and implement the "Shadow Processing" logic.

3. **Phase III (Refinement):** Deploy the "Human-in-the-Loop" verification UI and implement the Tiered Inference cost-saving model.

4. **Phase IV (Optimization):** Release the Waste Analytics engine and PII-stripping privacy layers for full production readiness.

PROJECTGENAI